

컴퓨터비전_인공지능(AI) 개론

1. 딥러닝에 필요한 파이썬의 개념

개념적 이해

딥러닝 학습 구조 이해
(순전파 → 손실 → 역전파 → 업데이트)

PyTorch 모델 구조 (함수/클래스) 파악

손실함수 · 역전파 · **Optimizer**의 역할과 위치 이해

실무적 적용

컨테이너 타입 변수의 안전한 처리(참조와 복사 개념)

실습 환경 구성 안내(**Colab** 또는 **VSCode+Conda**)

인간과 컴퓨터 학습과정



인간 학생의 학습

- 1  문제집 풀기
개념 학습 및 연습
- 2  채점 하기
정답과 비교
- 3  피드백 확인
틀린 이유 분석
- 4  다시 풀기
이해 후 개선 (반복)



컴퓨터(딥러닝)의 학습

- 1  순전파 (**Forward**)
데이터를 보고 예측
- 2  손실 계산 (**Loss**)
정답과 비교
- 3  오차 측정
얼마나 틀렸는지 수치화
- 4  역전파/업데이트
파라미터 수정 (최적화)

"채점하기"

컴퓨터의 답과 정답을 비교해서 얼마나 틀렸는지를 숫자로 표현합니다.

$$\begin{array}{ccc} \boxed{5} & - & \boxed{4} = \boxed{1} \\ \text{정답} & & \text{예측} \quad \text{손실 (Loss)} \end{array}$$

💡 손실이 작을수록(0에 가까울수록) 모델의 성능이 개선된 것입니다.

"구조를 파이썬 코드로 이해"

복잡한 수식 유도나 구현 세부 사항보다는 딥러닝의 전체 구조를 파이썬 코드 레벨에서 이해하는 데 집중합니다.

함수와 합성 함수

딥러닝 모델을 거대한 합성 함수로 바라보고 이를 파이썬 함수로 표현하는 방법을 배웁니다.

미분의 개념

학습의 방향을 결정하는 미분의 역할을 이해하고, **PyTorch**와 어떻게 연결되는지 확인합니다.

클래스 구조

데이터와 기능을 묶는 클래스의 개념을 익히고, **PyTorch** 모델 정의 방식의 기초를 다집니다.

"틀린 이유를 거꾸로 추적"

결과(Loss)에서 시작하여 입력 방향으로 거슬러 올라가며, 각 단계의 파라미터가 오차에 얼마나 기여했는지를 계산합니다.

1

원인 분석

출력에서 입력 방향으로 역추적하여 오차의 원인을 파악

2

기울기 (Signal)

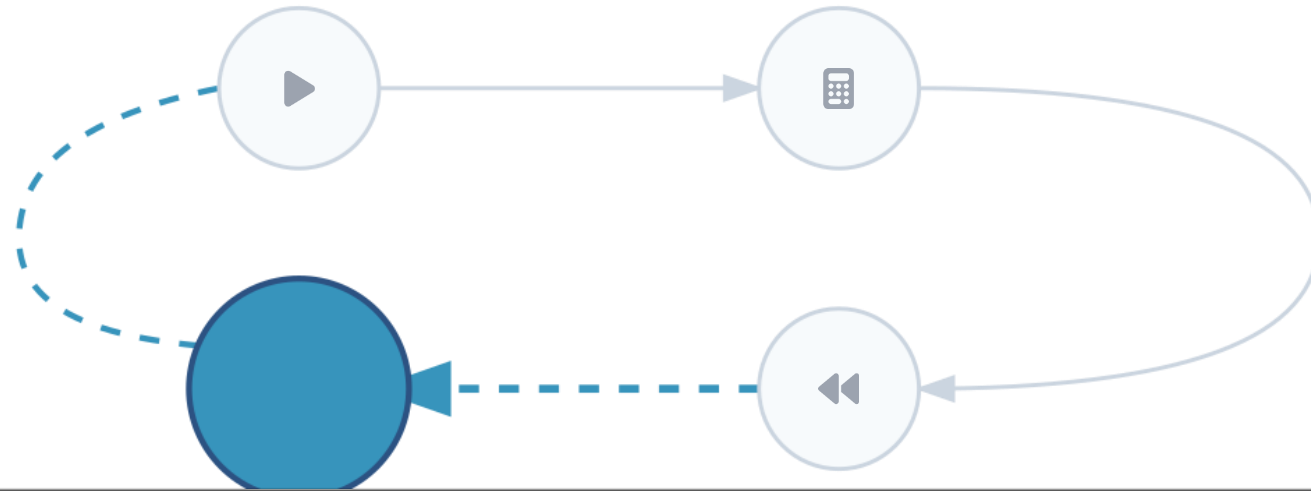
어디를 얼마나 고쳐야 하는지 알려주는 신호 (Gradient)를 계산

3

프레임워크 자동 처리

복잡한 미분 계산은 PyTorch (Autograd)가 자동으로 수행

파라미터 업데이트 과정



Optimizer의 역할

계산된 기울기(Gradient) 방향의 반대 방향으로 학습률(Learning Rate)만큼 이동하여 파라미터를 조정

$$\text{Param} = \text{Param} - (\text{lr} * \text{grad})$$

다음 스텝을 위한 새로운 가중치 결정



학습률이 너무 클 때

성큼성큼 이동하다가 최적점을 지나쳐 발산(Divergence)할 위험



학습률이 너무 작을 때

너무 조금씩 이동하여 학습 속도가 느려지고 최적점에 도달 못 함



점진적 개선

한 번에 완벽해지는 것이 아니라 반복을 통해 조금씩 나아집니다.

처음엔 무작위 추측이지만, 학습을 거듭하며 정답에 가까워집니다.



자동화

역전파와 최적화 과정을 컴퓨터가 자동으로 수행합니다.

복잡한 미분 계산을 개발자가 직접 할 필요가 없습니다.



일반화

학습하지 않은 새로운 데이터에도 성능을 유지합니다.

단순 암기가 아니라 데이터의 패턴과 규칙을 학습합니다.

PyTorch가 대신해주는 것

PyTorch는 복잡한 수학적 계산과 최적화 과정을 자동화하여, 개발자가 모델의 구조와 논리에만 집중할 수 있도록 돕습니다.

1. 자동화된 계산 및 도구

- 자동 미분 (**Autograd**) 순전파(**Forward**) 구조만 정의하면 미분과 역전파 과정을 자동으로 계산합니다.
- 내장 학습 도구 다양한 **Optimizer**와 손실 함수(**Loss Function**)가 모듈로 내장되어 있어 즉시 사용 가능합니다.

2. 고성능 연산 지원

- 텐서(**Tensor**) 연산 **NumPy**와 유사한 문법을 제공하여 사용이 쉽고, 대규모 행렬 연산에 최적화되어 있습니다.
- **GPU** 가속 복잡한 설정을 최소화하고 코드 한 줄로 **GPU**를 활용한 고속 연산을 지원합니다.

"수학적 구현보다 모델링 아이디어 자체에 집중할 수 있는 환경을 제공합니다."

파이토치에서 딥러닝 모델을 학습시킬 때, 다음과 같은 표준화된 5단계 절차를 반복합니다.



💡 평가(Evaluation) 시에는?

학습이 아닌 평가 단계에서는 기울기 계산이 불필요하므로, `with torch.no_grad():` 블록을 사용하여 불필요한 연산을 방지하고 메모리를 절약합니다.

컨테이너 타입 변수란?

리스트(list), 딕셔너리(dict), 넘파이 배열(numpy array)은 데이터를 담는 '그릇'이자 데이터를 가리키는 '참조(Reference)'입니다.



참조형 컨테이너

변수는 실제 값이 아닌 데이터가 위치한
메모리 주소를 저장

x -> [Memory 0x1234]



NumPy Array

대용량 수치 연산의 표준 컨테이너
(고속 연산, 메모리 효율)

Core Standard



Deep Learning

데이터 전처리 및 배치(Batch) 구성의
핵심 기반

Tensor Input

컨테이너의 함정: 값 공유(얇은 복사) 문제

컨테이너 변수를 대입(=)하면 값이 복사되는 것이 아니라, 같은 메모리 주소를 가리키게 됩니다.

⚠ 문제 발생: 얇은 복사 위험



```
y = x # 참조 복사
x[0] = 100 # 원본 변경
-> y도 [100, 7, 9]로 변함!
```

✖ 의도치 않은 데이터 변경

✓ 초기 상태: 정상 배열 생성



```
# 넘파이 배열 x 생성
x = np.array([5, 7, 9])
-> 정상적으로 배열 생성됨
```

i 초기 데이터 안전함

Critical Issue: 이러한 특성은 데이터 전처리 과정에서 원본 데이터가 훼손되는 심각한 버그를 유발할 수 있습니다.

copy()로 안전하게 다루기: 깊은 복사(Deep Copy)

변수 대입 시 `copy()` 메서드를 사용하면 새로운 메모리 공간을 확보하여 독립적인 복사본을 생성합니다.

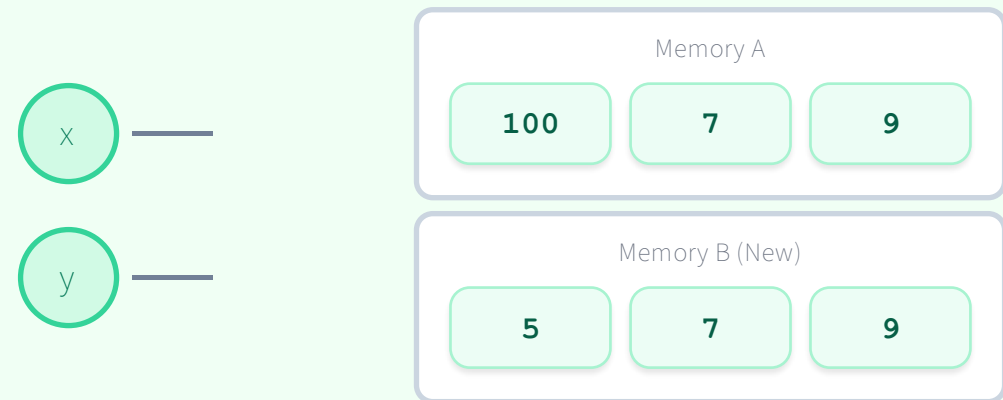
⚠ 얕은 복사 (참조)



```
y = x # 같은 주소 참조  
x[0] = 100  
-> y도 함께 변경됨 [100, 7, 9]
```

❌ 데이터 오염 위험

✅ 깊은 복사 (copy)

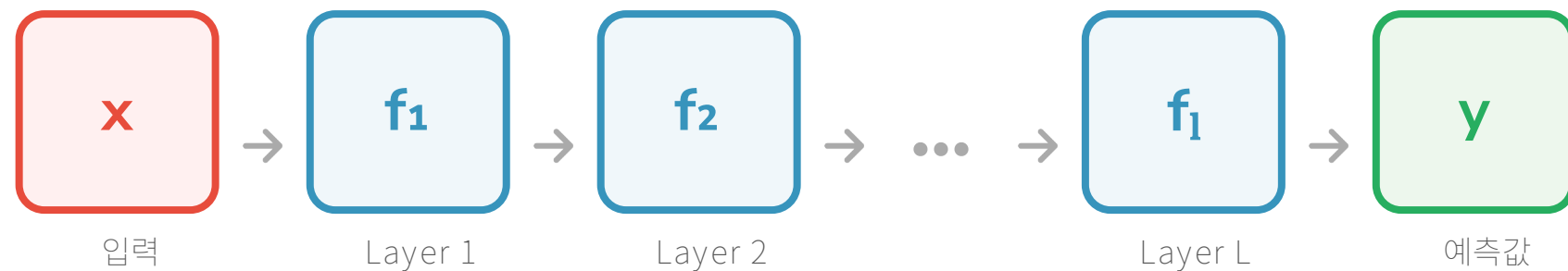


```
y = x.copy() # 새 메모리  
x[0] = 100  
-> y는 원본 유지 [5, 7, 9]
```

Key Benefit: 원본 데이터를 보존하면서 안전하게 실험할 수 있어 데이터 무결성과 실험 재현성을 확보합니다.

"함수들의 블록 조립"

딥러닝 모델은 수많은 함수(Layer)들이 순차적으로 연결된 구조입니다.
입력 데이터가 여러 층의 함수를 통과하면서 최종 결과로 변환됩니다.



$$y = f_1(\dots f_2(f_1(x)))$$

💡 이러한 합성 구조를 이해하면,
역전파(Backpropagation)가 이 과정을 거꾸로 거슬러 올라가는 것임을 직관적으로 알 수 있습니다.

파이썬으로 합성 함수 표현

수학적 합성 함수 $y = f_3(f_2(f_1(x)))$ 의 구조를 파이썬 함수로 직관적으로 구현할 수 있습니다.

```
def f1(x): return x**2 # 제곱
def f2(x): return 2*x # 2배
def f3(x): return x + 2 # +2 더하기
x = 3
# 합성 함수 실행: f1(3)=9 -> f2(9)=18 -> f3(18)=20
y = f3(f2(f1(x)))
print(y) # 출력: 20
```

딥러닝 모델의 순전파(**Forward Propagation**) 과정은 이러한 합성 함수의 연쇄적인 실행과 동일합니다.

"어느 방향으로 고쳐야 하는가?"

미분(기울기)은 학습의 나침반 역할을 합니다. 손실을 줄이기 위해 가중치를 어떻게 조절할지 알려줍니다.



방향과 크기

손실을 줄이기 위해
가중치를 키울지 줄일지,
얼마나 바꿀지 결정합니다.



민감도 (Sensitivity)

입력값이 조금 변할 때
결과(손실)가 얼마나
민감하게 변하는지 나타냅니다.



최적값 탐색

기울기가 0이 되는 지점,
즉 손실이 가장 낮은
최적값을 찾아갑니다.

중앙 차분 (Central Difference)

함수의 변화율을 근사하기 위해 x 를 중심으로 아주 작은 간격 h 의 양쪽 지점을 사용합니다.

공식 (Formula)

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$



오차 감소

전진/후진 차분보다 더 정확함



자동화

실무에선 프레임워크가 자동 처리



딥러닝에서는 기울기(Gradient)를 통해 오차를 줄이는 방향을 찾습니다.

수치미분 구현

파이썬의 고차 함수(Higher-order function) 개념을 활용하여, 어떤 함수든 미분해주는 **fdiff** 함수를 구현할 수 있습니다.

```
# 함수를 인자로 받아 도함수를 반환하는 함수 정의
def fdiff(f):
    def diff(x):
        h = 1e-6 # 0.000001 (매우 작은 값)
        # 중앙 차분 공식 적용: (f(x+h) - f(x-h)) / 2h
        return (f(x+h) - f(x-h)) / (2*h)
    return diff
# 테스트: f(x) = 2x^2 + 2
f = lambda x: 2*x**2 + 2
f_prime = fdiff(f) # 도함수 생성
# x = 3.0에서의 기울기 계산 (실제값 12)
print(f_prime(3.0)) # 출력: 12.0000000000...
```

이 코드는 미분의 원리를 이해하기 위한 데모입니다. 실제 딥러닝에서는 PyTorch의 자동 미분(Autograd)을 사용합니다.

PyTorch Autograd 미리보기

직접 수치 미분을 구현할 필요 없이,

PyTorch의 **Autograd**(자동 미분) 기능을 사용하면 복잡한 수식도 자동으로 미분할 수 있습니다.

```
import torch
# requires_grad=True: 추적 시작
x = torch.tensor(3.0, requires_grad=True)
y = 2 * x**2 + 2 # 그래프 생성
y.backward() # 자동 미분
print(x.grad) # 미분값 출력
```



```
Terminal
user@pytorch:~$ python
train.py
tensor(12.)
# 수치 미분 결과 (12.0)와
# 정확히 일치함!
```

Key Point: 우리가 앞서 배운 수치 미분의 원리가 **PyTorch** 내부에서 자동 미분으로 구현되어 있습니다.

클래스(Class) 개념 정리

PyTorch 딥러닝 모델은 모두 `nn.Module`을 상속받는 클래스 구조를 가집니다.

이를 이해하기 위한 객체지향(OOP)의 핵심 4대 요소를 정리합니다.

클래스 (Class)

객체를 만들기 위한 '설계도'입니다.
(예: 붕어빵 틀, 자동차 설계 도면)

인스턴스 (Instance)

설계도를 바탕으로 생성된 '실체'입니다.
(예: 팔 붕어빵, 실제 생산된 자동차)

속성 (Attribute)

객체가 가지는 '상태'나 '데이터'입니다.
(예: `self.weights`, `self.bias`)

메서드 (Method)

객체가 수행하는 '동작'이나 '함수'입니다.
(예: `forward()`, `backward()`)

Point 클래스 예제

2차원 좌표 평면 위의 점을 나타내는 **Point** 클래스를 정의하고, 메서드를 통해 상태를 변경하는 예제입니다.

```
class Point:
    def __init__(self, x, y): # 생성자: 인스턴스 초기화
        self.x = x
        self.y = y
    def move(self, dx, dy): # 메서드: 좌표 이동
        self.x += dx
        self.y += dy
    def coord(self): # 메서드: 현재 좌표 반환
        return (self.x, self.y)
p = Point(2, 3) # 인스턴스 생성 (x=2, y=3)
p.move(1, -1) # 이동 (x+1, y-1)
print(p.coord()) # 결과: (3, 2)
```

i 클래스를 사용하면 데이터(**x, y**)와 기능(**move, coord**)을 하나의 객체(**p**)로 묶어 관리할 수 있어 코드가 구조화됩니다.

호출 가능한 클래스 (__call__)

클래스 내부에 `__call__` 메서드를 정의하면, 인스턴스를 마치 함수처럼 호출하여 사용할 수 있습니다.

```
# 함수처럼 동작하는 클래스 H 정의
class H:
    def __call__(self, x):
        return 2 * x + 2
# H 클래스의 인스턴스 생성
h = H()
# 인스턴스를 함수처럼 호출 (h(3) >> __call__ >> 실행)
result = h(3)
print(result)
# 출력: 8
```

Tip:

PyTorch의 모든 신경망 모듈(`nn.Module`)은 이 방식을 사용합니다.

따라서 모델 사용 시 `model.forward(x)`가 아닌 `model(x)` 형태로 데이터를 전달합니다.

우리가 정의한 클래스와 함수 구조는 PyTorch의 핵심인 `nn.Module`과 동일한 형태를 가집니다.

1. 모델 구조 정의 (`nn.Module`)

```
class MyModel(nn.Module):  
    def __init__(self):  
        super().__init__()  
  
        # 필요한 재료(Layer) 준비  
  
    def forward(self, x):  
        # 순전파 과정 정의  
        return x
```

2. 학습 실행 (Training Loop)

```
# 1. 순전파 및 손실 계산  
pred = model(x)  
loss = criterion(pred, y)  
  
# 2. 역전파 (기울기 계산)  
loss.backward()  
  
# 3. 파라미터 업데이트  
optimizer.step()
```

다음 차시부터는 이 구조를 바탕으로 실제 데이터를 학습시키는 전체 파이프라인을 구현합니다.

 학습 사이클

순전파 (Forward) → 손실 계산 (Loss) → 역전파 (Backward)
→ 파라미터 업데이트

* 이 과정을 반복하며 모델이 점진적으로 개선됩니다.

 PyTorch의 역할

- Autograd: 복잡한 미분 계산 자동화
- Optimizer: 효율적인 파라미터 최적화
- Tensor: 고속 행렬 연산 및 GPU 가속 지원

 파이썬 핵심 개념

- 컨테이너: 데이터 관리 (List, Array)
- 합성 함수: 딥러닝 모델의 기본 구조
- 클래스(OOP): 데이터와 기능을 묶는 설계도

 실습 환경

- Google Colab: 별도 설치 없이 GPU 사용 가능 (권장)
- VSCode + Conda: 로컬 개발 환경 구축 시
(실습 코드 .ipynb 파일 활용)